Name : S. Akshitha
Reg. no: 192521390
Course : Datastructure
Course code: CSA0313
Date: 27/7/26.

Analytical tool:

1. Given Array

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

First we need to find mid value,

We know that

$$Mid = \frac{low + high}{2}$$

first lowest value = 66 ⇒ Index 0

highest value = 888 ⇒ Index 9

(i) Case = key = 11

L ⇒ Low

4 ⇒ High

L = 0

H = 4

$Mid = \frac{0+9}{2} = 4.5 = 4$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

now, we used to change the high value

high = mid − 1 = 4 − 1 = 3

L = 0

H = 3

$mid = 0 + 3/2 = 3/2 = 1.5 = 1$

Element = 111

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

The given element is found at index 1

Case 2 : key = 888

L = 0

H = 3

Mid = $\frac{low + high}{2} = \frac{0+9}{2} = 4.5 = \boxed{4}$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

888 7432

now the given value is greater than middle element

So,

low = mid + 1

Low = 5

High = 9

Mid = $\frac{5+9}{2} = \frac{14}{2} = \boxed{7}$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

8887649

So, Low = mid + 1 = 7 + 1 = $\boxed{8}$

L = 8

H = 9

Mid = $\frac{L+H}{2} = \frac{8+9}{2} = \frac{17}{2} = 8$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

8887 777

L = Mid + 1 = 8 + 1 = 9

L = 9

H = 9

Mid = $\frac{9+9}{2} = \frac{18}{2} = \boxed{9}$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

888 = 888

The given element is found at index [9]

Case 2: key = 999:

L = 0

H = 9

M = $\frac{0+9}{2} = 4.5 = \boxed{4}$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

999 7432

So,

L = Mid + 1 = 4 + 1 = 5

Mid = $\frac{5+9}{2} = \frac{14}{2} = 7$

66, 111, 212, 343, 423, 543, 643, 649, 777, 888

999 7432

So,

L = mid + 1 = 4 + 1 = 5

Mid = $\frac{5+9}{2} = \frac{14}{2} = 7$

66, 111, 212, 343, 423, 543, 643, 649, 777, 888

9997649

low = mid + 1 = 7 + 1 = 8

L = 8

H = 9 Mid = $\frac{8+9}{2} = \frac{17}{2} = 8$

Low = mid + 1 = 7 + 1 = 8

L = 8
H = 9

$Mid = \frac{8+9}{2} = \frac{17}{2} = 8$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

9997777

L = Mid + 1 = 8 + 1 = 9

$Mid = \frac{9+9}{2} = \frac{18}{2} = \boxed{9}$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

999 > 888

Finally,

The given element is not found

---

2. given array

66	111	212	323	432	543	643	649	777	888
0	1	2	3	4	5	6	7	8	9

Index / element

Key = 111

Low	High	Mid	Mid element	Result
0	9	4	432	111 < 432 → high = 3
0	3	1	111	found in 2nd iterations

Index = $\boxed{1}$

ii) Key = 888

Low	High	mid	mid element	Result
0	9	4	432	888 > 432
5	9	7	649	888 < 649
8	9	8	777	888 > 777
9	9	9	888	found

found at index [9]

iii) Key = 999

Low	High	mid	midelement	result
0	9	4	432	999 > 432 ✓
5	9	7	649	999 > 649
8	9	8	777	999 > 777
9	9	9	888	999 > 888

Low(10) > High(9)

∴ Element is not found

Analysis of Binary search

→ It works on sorted arrays

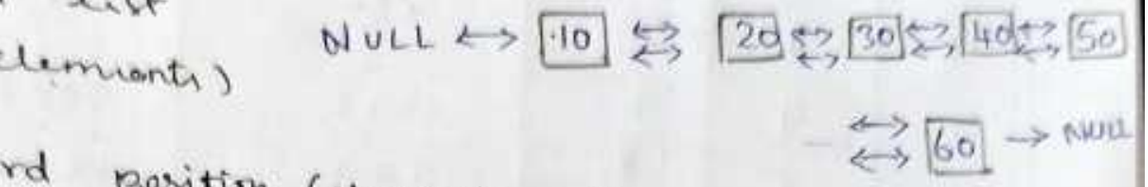
Time complexity

* Best case = O(1)
* Average case = O(log 2^n)
* Worst Case = O(log 2^n)
* Space Complexity = O(1)

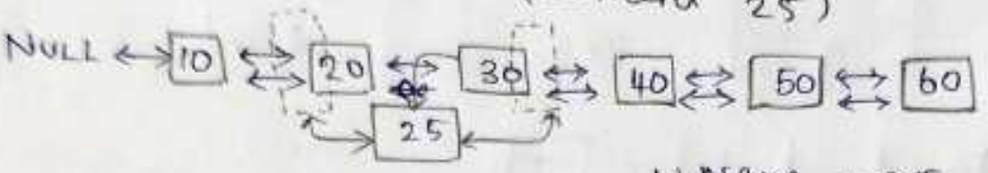
3] Doubly linked list

a]. Insert an element at 3rd position

Initial list (6 elements)



Insert 3rd position (element 25)

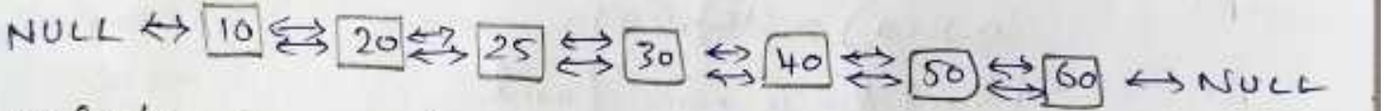


1. New = 25
2. New → prev = 20
3. New → Next = 30
4. 20 → next = New
5. 30 → prev = new

After insertion:

b] Delete the element at 2nd position:

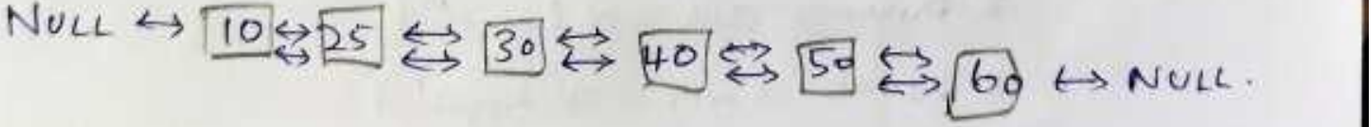
list before deletion:



Delete 20 (2nd position)

1). 10 → next = 25
2. 25 → prev = 10
3. Delete 20

After deletion:



Two stacks in a single array.

Array size = 10

Index :	0	1	2	3	4	5	6	7	8	9
Initial :	-	-	-	-	-	-	-	-	-	-

Stacks 1 grows from left →

Stacks 2 grows from right

Example :

10	20	30	-	-	-	-	90	80	70

↑ Top 1

↑ Top 2

→ Dynamic allocation uses space efficiently

→ Both push and pop operations are O(1)

Overflow cond : Top 1+1 = Top 2

comparison :	Fixed partitioning	Dynamic allocation.
Memory division	Array divided equally into two parts	Both stack share entire array
Space utilization	Lower	higher
flexibility	one stack may overflow even if other has space	only when Top 1+1 = Top 2 (array full)
Push pop complexity	O(1)	O(1).

4]. B − (C / D + (E % F * G / H) *])

Symbol	Postfix string	Stack
B	B	
−	B	−
(	B	− (
C	B C	− (
/	B C	− (/
D	B C D	− (/ .
+	B C D /	− (/ +
E	B C D / E	− (+ (
%	B C D / E	− (+ (%
F	B C D / E F %	− (+ (%
*	B C D / E F % .	− (+ (*
G	B C D / E F % . G	− (+ (*
)	B C D / E F % . G *	− (+ (*
/	B C D / E F % . G *	− (+
H	B C D / E F % G * H	− (+ /
)	B C D / E F % G * H / +	− (+ /
*	B C D / E F % G * H / +	−
J	B C D / E F % G * H / +]	− *
End	B C D / E F % G * H / +] −	− *

5]. Circular queue (size = 5)

Queue size = 5

position = 0, 1, 2, 3, 4

operations:

Enqueue (20) Enqueue (10), enqueue (30, Dequeue (1, enqueue (80) Dequeue (1, enqueue 90), enque (100), Dequeue (1, Dequeue (1, enqueue 60), enqueue (90).

Step	Operation	front	rear	Queue Status
Initial	Queue empty	-1	-1	[-, -, -, -, -]
1	Enqueue (20)	0	0	[20, -, -, -, -]
2	enqueue (10)	0	1	[20, 10, -, -, -]
3	enqueue (30)	0	2	
4	Dequeue -7 remove 20	1	2	[20, 10, 30, -, -]
		1	3	[-, 10, 30, -, -]
5	enqueue (80)	2	3	[-, 10, 30, 80, -]
6	Dequeue	2	4	[-, -, 30, 80, 90]
7	enqueue (90)	2	0	[100, -, 30, 80, 90]
8	enqueue (100)	3	0	[100, -, -, 80, 90]
9	Dequeue (.)	4	0	[100, -, -, -, 90]
10	Dequeue (.)	4	1	[100, 60, -, -, 90]
11	enqueue (60)	4	2	[100, 60, 90, -90]
12	enqueue (90)			

9] 9 3 8 4 / * +

Symbol	Stack operation	Stack
9	push 9	9
3	push 3	9, 3
8	push 8	9, 3, 8
4	push 4	9, 3, 8, 4
/	8 ÷ 4 = 2	9, 3, 2
*	3 x 2 = 6	9, 6
+	9 + 6 = 15	15
		ans = 15

b] postfix expression :-
6 2 + 6 3 / *

symbol	stack operation	stack
6	push 6	6
2	push 2	6, 2
+	6 + 2 = 8	8
6	push 6	8, 6
3	push 3	8, 6, 3
/	6 ÷ 3 = 2	8, 2
*	8 x 2 = 16	16

ans = 16

Position	0	1	2	3	4
element	100	60	90	Smp	90

front element = 4 = 90 (position 4)

Rear = 2 = 90 (position 2).